Taller Playwright-e2e

Taller evaluativo: pruebas End-to-End con Playwright y CI/CD

Enlace repositorio

https://github.com/AdriMatinez/taller-playwright-e2e-A00401480-A00400842.git

Integrantes

Johan Stiven Guzmán — A00401480
Adri Jhoanny Martinez Murillo — A00400842

Docente

Juan José Valencia

Asignatura

Ingesoft V

Cali, Valle Del Cauca - Colombia
20 de octubre de 2026

Punto 2.1 – Fase 1: Configuración de Playwright, tooling y multi-navegador

¿Qué hicimos?

Partimos del repositorio base del taller, que ya traía el servidor Express, el frontend en React con Vite y una configuración casi completa de Playwright. Antes de escribir pruebas, levantamos ambos servicios con npm run dev: la API quedó escuchando en http://localhost:8080 y el frontend (Vite) en http://localhost:3000, como lo exige el enunciado.

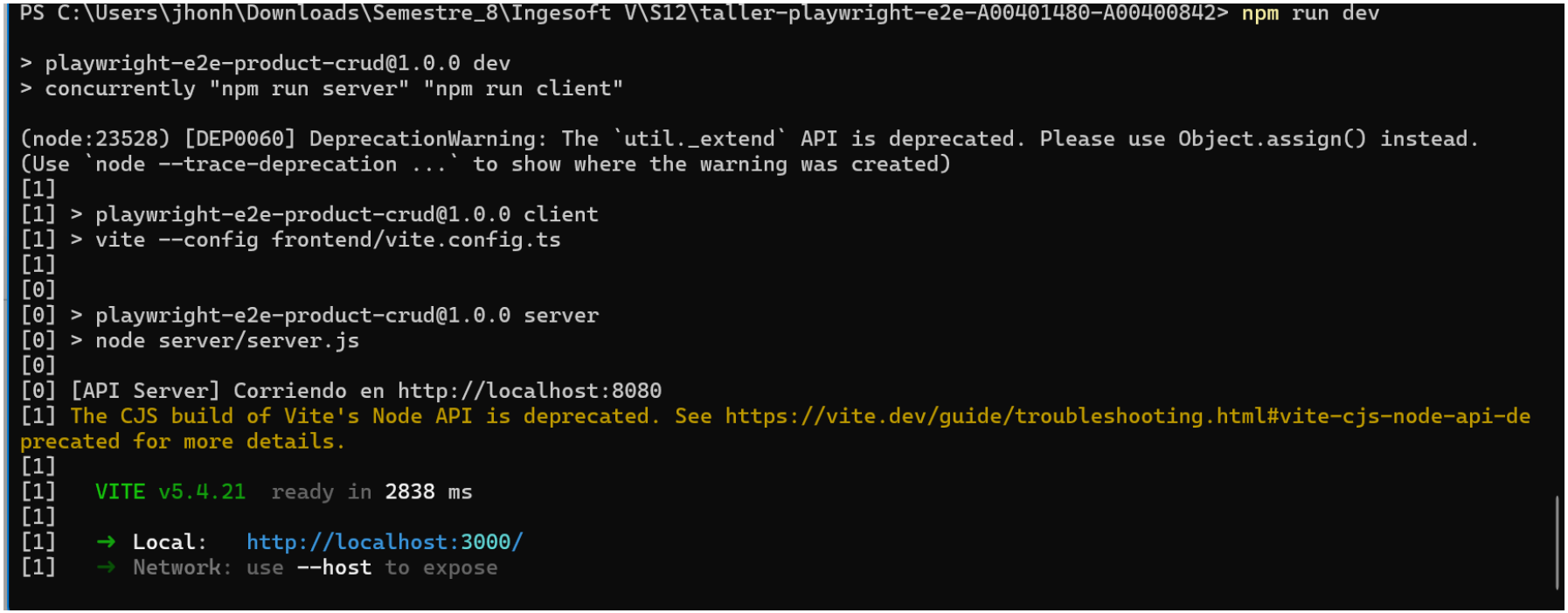


Figura 1. *Levantamiento de los servicios con npm run dev: API en http://localhost:8080 y frontend con Vite en http://localhost:3000.*

Luego ejecutamos la suite con npm run test:e2e. En el repositorio base, ProductPage.ts y products.spec.ts solo contenían comentarios TODO, de modo que sus pruebas habrían pasado en verde sin verificar nada. Por eso los completamos y también agregamos LoginPage.ts, para contar desde el inicio con pruebas reales corriendo en los tres navegadores.

Con esta configuración quedan cubiertos los parámetros del punto: baseURL, timeout de 30 segundos por prueba, política de reintentos en CI (retries: process.env.CI ? 2 : 0), diagnóstico con trace, screenshot y video, los tres proyectos (Chromium, Firefox y WebKit) y los scripts de package.json. Además, npm install generó el archivo package-lock.json, que necesitamos más adelante porque el pipeline usa npm ci.

Evidencia

La ejecución terminó con **12 pruebas aprobadas** (4 pruebas × 3 navegadores), lo que confirma que la configuración multi-navegador funciona.

```
PS C:\Users\jhonh\Downloads\Semestre_8\Ingesoft V\S12\taller-playwright-e2e-A00401480-A00400842> npm run test:e2e

> playwright-e2e-product-crud@1.0.0 test:e2e
> playwright test


Running 12 tests using 2 workers

  ✓   1 …:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (9.4s)
  ✓   2 ….ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (9.2s)
  ✓   3 ….ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (1.9s)
  ✓   4 ….ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (2.0s)
  ✓   5 …13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (23.2s)
  ✓   6 …ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (23.8s)
  ✓   7 ….ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (4.6s)
  ✓   8 ….ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (5.0s)
  ✓   9 …ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (14.1s)
  ✓  10 …13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (15.9s)
  ✓  11 …ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (11.2s)
  ✓  12 …ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (11.9s)

  12 passed (1.4m)

To open last HTML report run:

  npx playwright show-report

PS C:\Users\jhonh\Downloads\Semestre_8\Ingesoft V\S12\taller-playwright-e2e-A00401480-A00400842>
```

Figura 2. *Primera ejecución de npm run test:e2e: 12 pruebas aprobadas (4 pruebas × 3 navegadores) con ProductPage.ts y products.spec.ts completados.*

Punto 2.2 – Fase 2: Page Object Model (POM) y flujos críticos de negocio

¿Qué hicimos?

El repositorio base no tenía login, carrito ni checkout, así que esta fase exigió extender tanto el backend como el frontend, no solo las pruebas. Mantuvimos el stack del repositorio (Express en el backend y React en el frontend) y agregamos al servidor el inicio de sesión y la creación de órdenes (POST /api/v1/orders), y al frontend las vistas de login, catálogo con buscador, carrito y checkout, con roles y etiquetas accesibles.

En e2e/pages/ creamos las clases LoginPage.ts, CatalogPage.ts y CheckoutPage.ts, que usan localizadores semánticos (getByRole, getByLabel y getByPlaceholder) y no contienen aserciones; estas quedan en los specs como aserciones web-first.

La suite order-lifecycle.spec.ts cubre el camino feliz (login, catálogo, carrito, checkout y confirmación con el identificador de la orden), la actualización de badge y subtotal al agregar varias unidades de un mismo producto, el intento de acceso con credenciales inválidas (alerta visible sin salir de /login) y la restricción de que el botón de confirmar permanezca deshabilitado con el carrito vacío.

Evidencia

Al ejecutar nuevamente npm run test:e2e obtuvimos **24 pruebas aprobadas** (8 pruebas × 3 navegadores): las 4 del ciclo de vida de la orden y las 4 de productos en Chromium, Firefox y WebKit.

```
PS C:\Users\jhonh\Downloads\Semestre_8\Ingesoft V\S12\taller-playwright-e2e-A00401480-A00400842> npm run test:e2e

> playwright-e2e-product-crud@1.0.0 test:e2e
> playwright test


Running 24 tests using 2 workers

  ✓   1 …, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (3.8s)
  ✓   2 …tálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (3.8s)
  ✓   3 …la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (2.7s)
  ✓   4 … catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (2.8s)
  ✓   5 …:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (1.4s)
  ✓   6 ….ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (1.3s)
  ✓   7 ….ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (1.7s)
  ✓   8 ….ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (1.9s)
  ✓   9 …tálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (9.8s)
  ✓  10 … catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (10.0s)
  ✓  11 …la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (4.0s)
  ✓  12 … catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (3.9s)
  ✓  13 …:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (3.5s)
  ✓  14 ….ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (3.6s)
  ✓  15 ….ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (3.3s)
  ✓  16 ….ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (3.7s)
  ✓  17 …álogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (10.4s)
  ✓  18 …, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (6.1s)
  ✓  19 …la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (7.8s)
  ✓  20 … catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (7.2s)
  ✓  21 …:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (6.9s)
  ✓  22 ….ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (6.0s)
  ✓  23 ….ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (7.7s)
  ✓  24 ….ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (7.3s)

  24 passed (1.2m)

To open last HTML report run:

  npx playwright show-report
```

Figura 3. *Ejecución de npm run test:e2e con los POM y el ciclo de vida de la orden: 24 pruebas aprobadas (8 pruebas × 3 navegadores).*

Punto 2.3 – Fase 3: Network Mocking y resiliencia

¿Qué hicimos?

Agregamos el archivo network-resilience.spec.ts, donde validamos la tolerancia a fallos del frontend con page.route() en tres escenarios: (1) un error HTTP 500 al crear la orden, para comprobar que la interfaz muestra el diálogo de reintento y sigue siendo usable; (2) la recuperación, en la que el reintento funciona cuando el servicio vuelve a responder; y (3) una latencia artificial de 3000 ms sobre GET /api/v1/products, verificando que el indicador de carga aparece durante la espera y se oculta al terminar.

Evidencia

Con las pruebas de red incorporadas, la suite completa pasó con **33 pruebas aprobadas** (11 pruebas × 3 navegadores).

```
PS C:\Users\jhonh\Downloads\Semestre_8\Ingesoft V\S12\taller-playwright-e2e-A00401480-A00400842> npm run test:e2e

> playwright-e2e-product-crud@1.0.0 test:e2e
> playwright test


Running 33 tests using 2 workers

  ✓   1 …lience.spec.ts:58:7 › Network Mocking y resiliencia del frontend › Recuperación: el reintento funciona cuando el servicio vuelve a responder (5.7s)
  ✓   2 …c.ts:29:7 › Network Mocking y resiliencia del frontend › Error HTTP 500 al crear la orden: muestra diálogo de reintento y la UI sigue usable (5.6s)
  ✓   3 …:7 › Network Mocking y resiliencia del frontend › Latencia de 3000 ms en GET /products: el spinner aparece y desaparece al terminar la carga (5.0s)
  ✓   4 … de la orden: login, catálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (2.8s)
  ✓   5 …vida de la orden: login, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (1.5s)
  ✓   6 …5:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (1.5s)
  ✓   7 …ida de la orden: login, catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (1.7s)
  ✓   8 …\tests\products.spec.ts:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (1.8s)
  ✓   9 …e2e\tests\products.spec.ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (1.5s)
  ✓  10 …e2e\tests\products.spec.ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (1.5s)
  ✓  11 …e2e\tests\products.spec.ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (1.5s)
  ✓  12 …ience.spec.ts:58:7 › Network Mocking y resiliencia del frontend › Recuperación: el reintento funciona cuando el servicio vuelve a responder (14.4s)
  ✓  13 ….ts:29:7 › Network Mocking y resiliencia del frontend › Error HTTP 500 al crear la orden: muestra diálogo de reintento y la UI sigue usable (15.0s)
  ✓  14 …:7 › Network Mocking y resiliencia del frontend › Latencia de 3000 ms en GET /products: el spinner aparece y desaparece al terminar la carga (7.0s)
  ✓  15 … de la orden: login, catálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (4.4s)
  ✓  16 …vida de la orden: login, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (4.6s)
  ✓  17 …5:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (4.1s)
  ✓  18 …ida de la orden: login, catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (6.9s)
  ✓  19 …\tests\products.spec.ts:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (9.0s)
  ✓  20 …e2e\tests\products.spec.ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (8.5s)
  ✓  21 …e2e\tests\products.spec.ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (7.0s)
  ✓  22 …e2e\tests\products.spec.ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (5.9s)
  ✓  23 ….ts:29:7 › Network Mocking y resiliencia del frontend › Error HTTP 500 al crear la orden: muestra diálogo de reintento y la UI sigue usable (13.5s)
  ✓  24 …ience.spec.ts:58:7 › Network Mocking y resiliencia del frontend › Recuperación: el reintento funciona cuando el servicio vuelve a responder (15.0s)
  ✓  25 …7 › Network Mocking y resiliencia del frontend › Latencia de 3000 ms en GET /products: el spinner aparece y desaparece al terminar la carga (13.4s)
  ✓  26 …de la orden: login, catálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (13.8s)
  ✓  27 …vida de la orden: login, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (8.2s)
  ✓  28 …5:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (5.6s)
  ✓  29 …ida de la orden: login, catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (7.7s)
  ✓  30 …\tests\products.spec.ts:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (9.0s)
  ✓  31 …e2e\tests\products.spec.ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (9.8s)
  ✓  32 …2e\tests\products.spec.ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (12.1s)
  ✓  33 …2e\tests\products.spec.ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (11.3s)

  33 passed (2.2m)
```

Figura 4. *Suite completa con network-resilience.spec.ts: 33 pruebas aprobadas (11 pruebas × 3 navegadores).*

Para el diagnóstico con Trace Viewer ejecutamos npm run test:e2e:trace, que equivale a npx playwright test --trace on. Las 33 pruebas volvieron a pasar y quedó registrada la traza de cada una.

```
PS C:\Users\jhonh\Downloads\Semestre_8\Ingesoft V\S12\taller-playwright-e2e-A00401480-A00400842> npm run test:e2e:trace

> playwright-e2e-product-crud@1.0.0 test:e2e:trace
> playwright test --trace on


Running 33 tests using 2 workers

  ✓   1 [chromium] › e2e\tests\network-resilience.spec.ts:58:7 › Network Mocking y resiliencia del frontend › Recuperación: el reintento funciona cuando el servicio vuelve a responder (9.0s)
  ✓   2 [chromium] › e2e\tests\network-resilience.spec.ts:29:7 › Network Mocking y resiliencia del frontend › Error HTTP 500 al crear la orden: muestra diálogo de reintento y la UI sigue usable (9.0s)
  ✓   3 [chromium] › e2e\tests\network-resilience.spec.ts:89:7 › Network Mocking y resiliencia del frontend › Latencia de 3000 ms en GET /products: el spinner aparece y desaparece al terminar la carga (4.9s)
  ✓   4 …e\tests\order-lifecycle.spec.ts:28:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (1.9s)
  ✓   5 …› e2e\tests\order-lifecycle.spec.ts:62:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (1.8s)
  ✓   6 [chromium] › e2e\tests\order-lifecycle.spec.ts:75:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (1.8s)
  ✓   7 … e2e\tests\order-lifecycle.spec.ts:84:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (2.4s)
  ✓   8 [chromium] › e2e\tests\products.spec.ts:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (2.3s)
  ✓   9 [chromium] › e2e\tests\products.spec.ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (2.1s)
  ✓  10 [chromium] › e2e\tests\products.spec.ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (2.2s)
  ✓  11 [chromium] › e2e\tests\products.spec.ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (2.2s)
  ✓  12 [firefox] › e2e\tests\network-resilience.spec.ts:29:7 › Network Mocking y resiliencia del frontend › Error HTTP 500 al crear la orden: muestra diálogo de reintento y la UI sigue usable (22.1s)
  ✓  13 [firefox] › e2e\tests\network-resilience.spec.ts:58:7 › Network Mocking y resiliencia del frontend › Recuperación: el reintento funciona cuando el servicio vuelve a responder (20.6s)
  ✓  14 [firefox] › e2e\tests\network-resilience.spec.ts:89:7 › Network Mocking y resiliencia del frontend › Latencia de 3000 ms en GET /products: el spinner aparece y desaparece al terminar la carga (7.7s)
  ✓  15 …e\tests\order-lifecycle.spec.ts:28:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (5.3s)
  ✓  16 …› e2e\tests\order-lifecycle.spec.ts:62:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (3.6s)
  ✓  17 [firefox] › e2e\tests\order-lifecycle.spec.ts:75:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (3.0s)
  ✓  18 … e2e\tests\order-lifecycle.spec.ts:84:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (3.4s)
  ✓  19 [firefox] › e2e\tests\products.spec.ts:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (3.3s)
  ✓  20 [firefox] › e2e\tests\products.spec.ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (3.0s)
  ✓  21 [firefox] › e2e\tests\products.spec.ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (3.3s)
  ✓  22 [firefox] › e2e\tests\products.spec.ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (4.1s)
  ✓  23 [webkit] › e2e\tests\network-resilience.spec.ts:29:7 › Network Mocking y resiliencia del frontend › Error HTTP 500 al crear la orden: muestra diálogo de reintento y la UI sigue usable (17.3s)
  ✓  24 [webkit] › e2e\tests\network-resilience.spec.ts:58:7 › Network Mocking y resiliencia del frontend › Recuperación: el reintento funciona cuando el servicio vuelve a responder (14.6s)
  ✓  25 [webkit] › e2e\tests\network-resilience.spec.ts:89:7 › Network Mocking y resiliencia del frontend › Latencia de 3000 ms en GET /products: el spinner aparece y desaparece al terminar la carga (10.0s)
  ✓  26 …\tests\order-lifecycle.spec.ts:28:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 1 (Camino feliz): login, catálogo, carrito, checkout y confirmación con ID de orden (14.5s)
  ✓  27 … e2e\tests\order-lifecycle.spec.ts:62:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 1b (Subtotales): varias unidades del mismo producto actualizan badge y subtotal (10.7s)
  ✓  28 [webkit] › e2e\tests\order-lifecycle.spec.ts:75:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 2 (Credenciales inválidas): muestra alerta y permanece en /login (7.6s)
  ✓  29 … e2e\tests\order-lifecycle.spec.ts:84:7 › Ciclo de vida de la orden: login, catálogo, carrito y checkout › Test 3 (Restricciones): el botón de confirmar está deshabilitado con el carrito vacío (7.4s)
  ✓  30 [webkit] › e2e\tests\products.spec.ts:13:7 › Suite E2E: Catálogo de Productos › Prueba 1 (Happy Path): Debe registrar un producto y reflejarlo en la tabla (7.1s)
  ✓  31 [webkit] › e2e\tests\products.spec.ts:26:7 › Suite E2E: Catálogo de Productos › Prueba 2 (Validación de Formulario): No permite registrar campos vacíos (5.6s)
  ✓  32 [webkit] › e2e\tests\products.spec.ts:33:7 › Suite E2E: Catálogo de Productos › Prueba 3 (Network Mocking): Manejo resiliente de error 500 del servidor (8.0s)
  ✓  33 [webkit] › e2e\tests\products.spec.ts:52:7 › Suite E2E: Catálogo de Productos › Prueba 4 (Eliminación): Debe eliminar un producto existente de la lista (5.9s)

  33 passed (2.2m)
```

Figura 5. *Ejecución de npm run test:e2e:trace (playwright test --trace on): 33 pruebas aprobadas con trazas habilitadas en Chromium, Firefox y WebKit.*

Después abrimos el reporte HTML de Playwright en http://localhost:9323. El resumen muestra 33 pruebas aprobadas, 0 fallidas, 0 intermitentes y 0 omitidas; cada resultado incluye el enlace View Trace, desde el cual se pueden revisar las acciones, el DOM y las llamadas de red de la prueba. Las siguientes capturas muestran el detalle por archivo de pruebas, con las etiquetas de los tres motores (chromium, firefox y webkit).

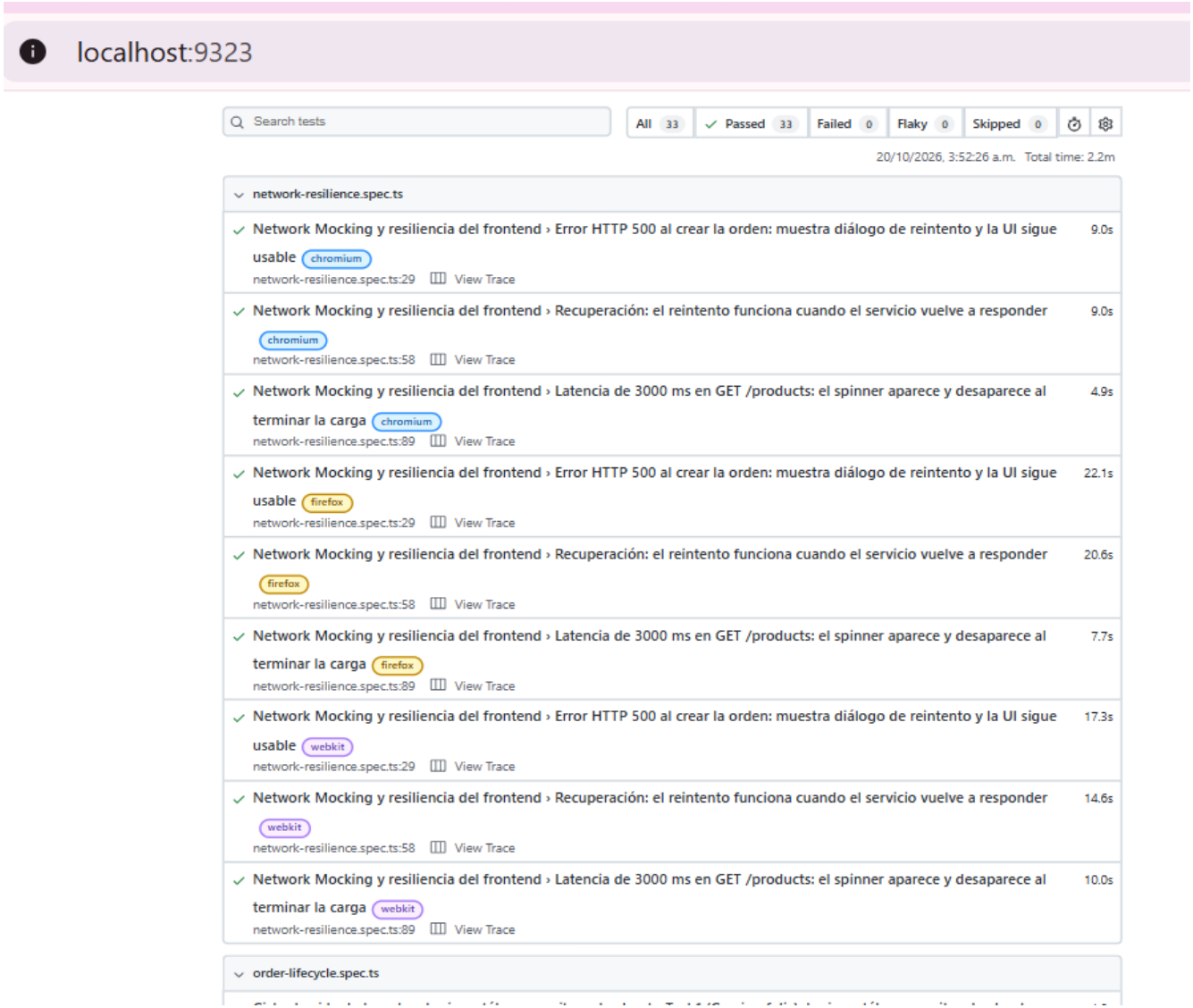


Figura 6. *Reporte HTML de Playwright: 33 pruebas aprobadas, 0 fallidas y 0 intermitentes; resultados de network-resilience.spec.ts en los tres navegadores.*

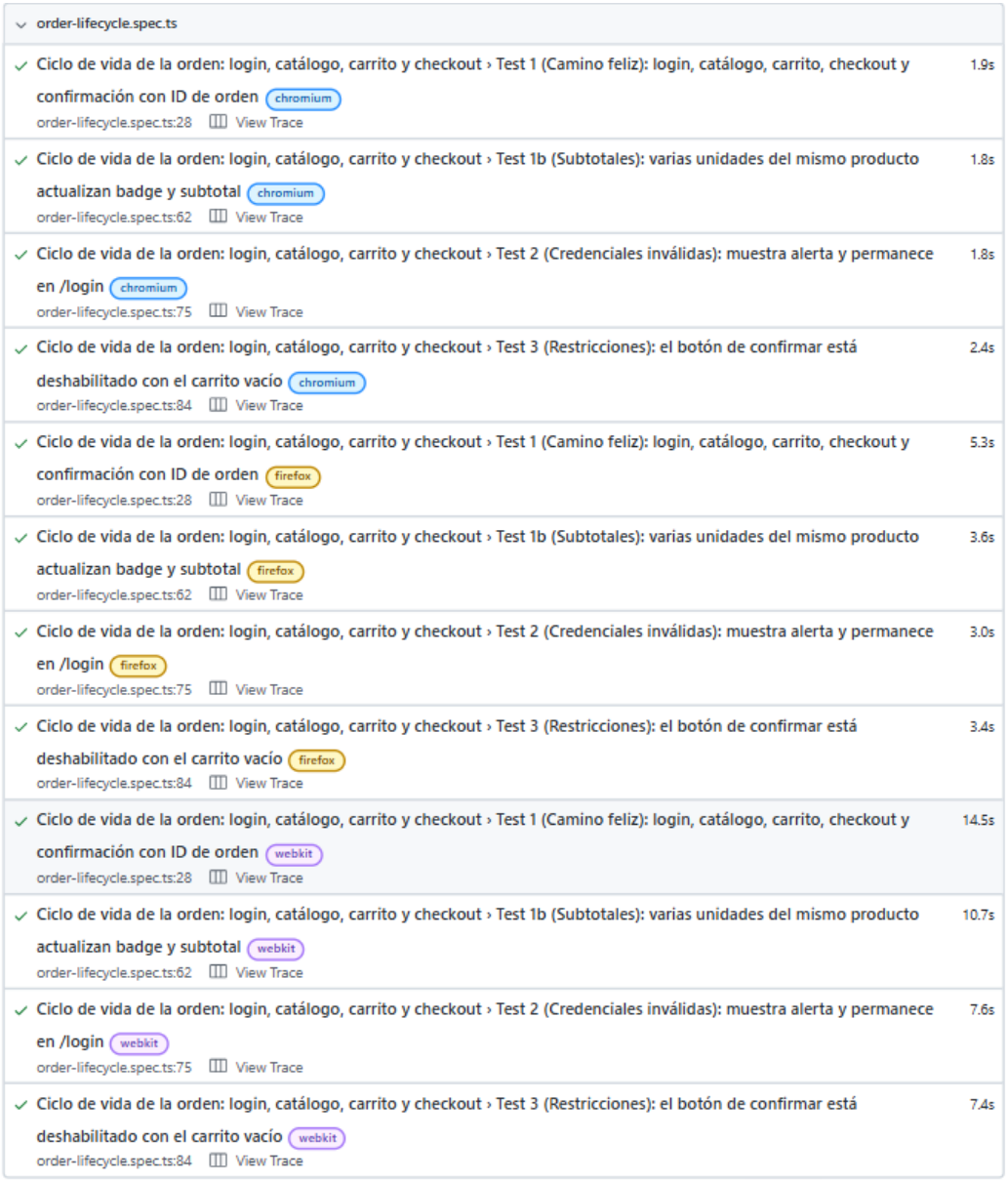


Figura 7. *Reporte HTML de Playwright: resultados de order-lifecycle.spec.ts (camino feliz, subtotales, credenciales inválidas y restricciones) en Chromium, Firefox y WebKit.*

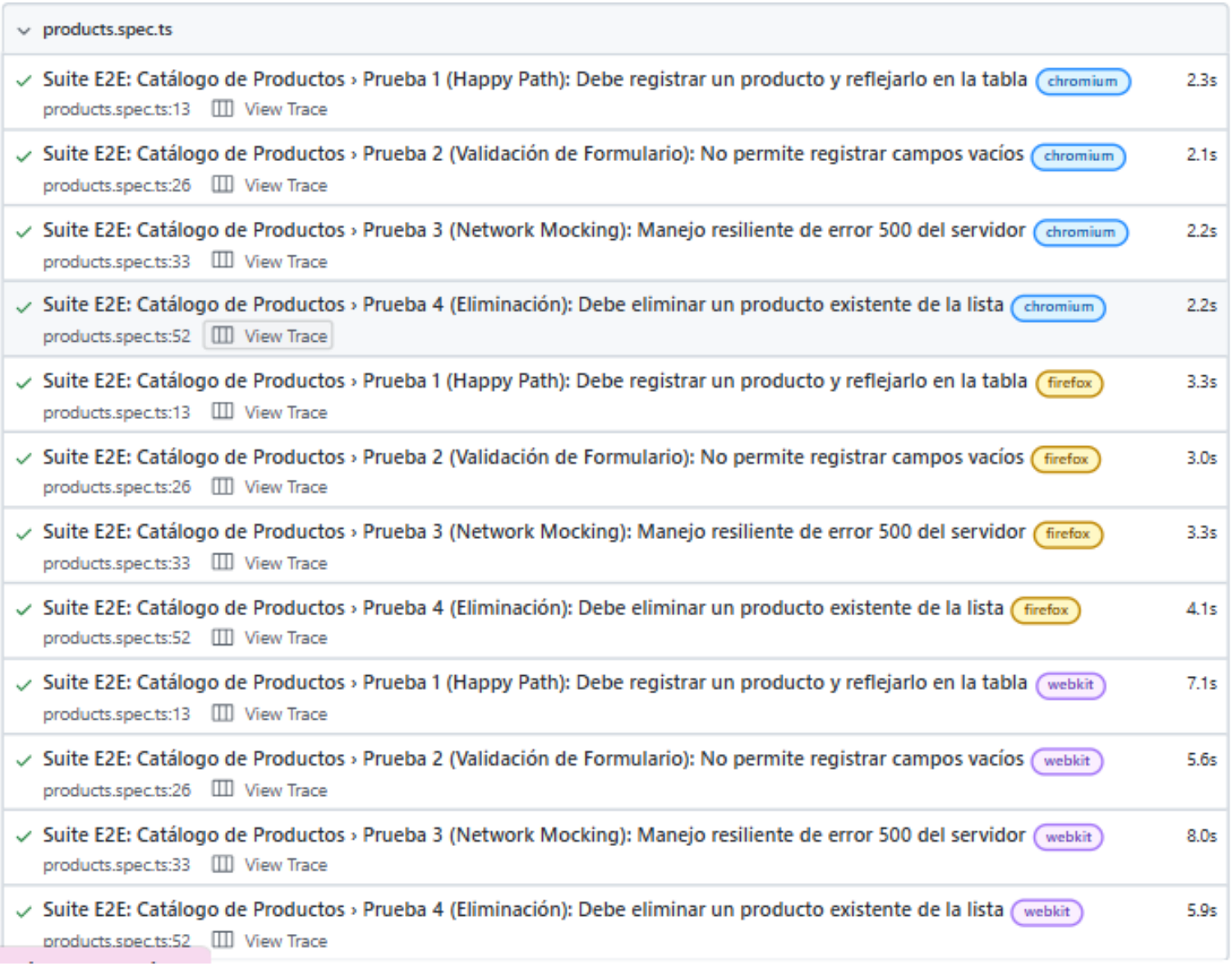


Figura 8. *Reporte HTML de Playwright: resultados de products.spec.ts en Chromium, Firefox y WebKit.*

Punto 2.4 – Fase 4: Pipeline declarativo en GitHub Actions

¿Qué hicimos?

Configuramos el flujo de integración continua en .github/workflows/e2e-playwright.yml con disparadores en push y pull_request, un job sobre ubuntu-latest con Node 20, instalación de dependencias con npm ci, instalación de los navegadores con npx playwright install --with-deps, ejecución headless de las pruebas y publicación del reporte HTML con actions/upload-artifact@v4, if: always() y retención de 30 días. Un paso adicional sube trazas, capturas y videos solo cuando hay fallos. Finalmente, subimos los cambios al repositorio.

Evidencia

El push disparó la ejecución **#1** del workflow E2E Playwright Suite, que terminó con éxito en 2 min 36 s. En el detalle del job Playwright E2E Multi-Browser se ven completados la configuración de Node.js 20, la instalación de dependencias, la instalación de los navegadores con dependencias del sistema (1 min 1 s), la ejecución de las pruebas en Chromium, Firefox y WebKit (1 min 12 s) y la subida del reporte HTML. El paso de trazas, capturas y videos aparece omitido porque no hubo fallos.

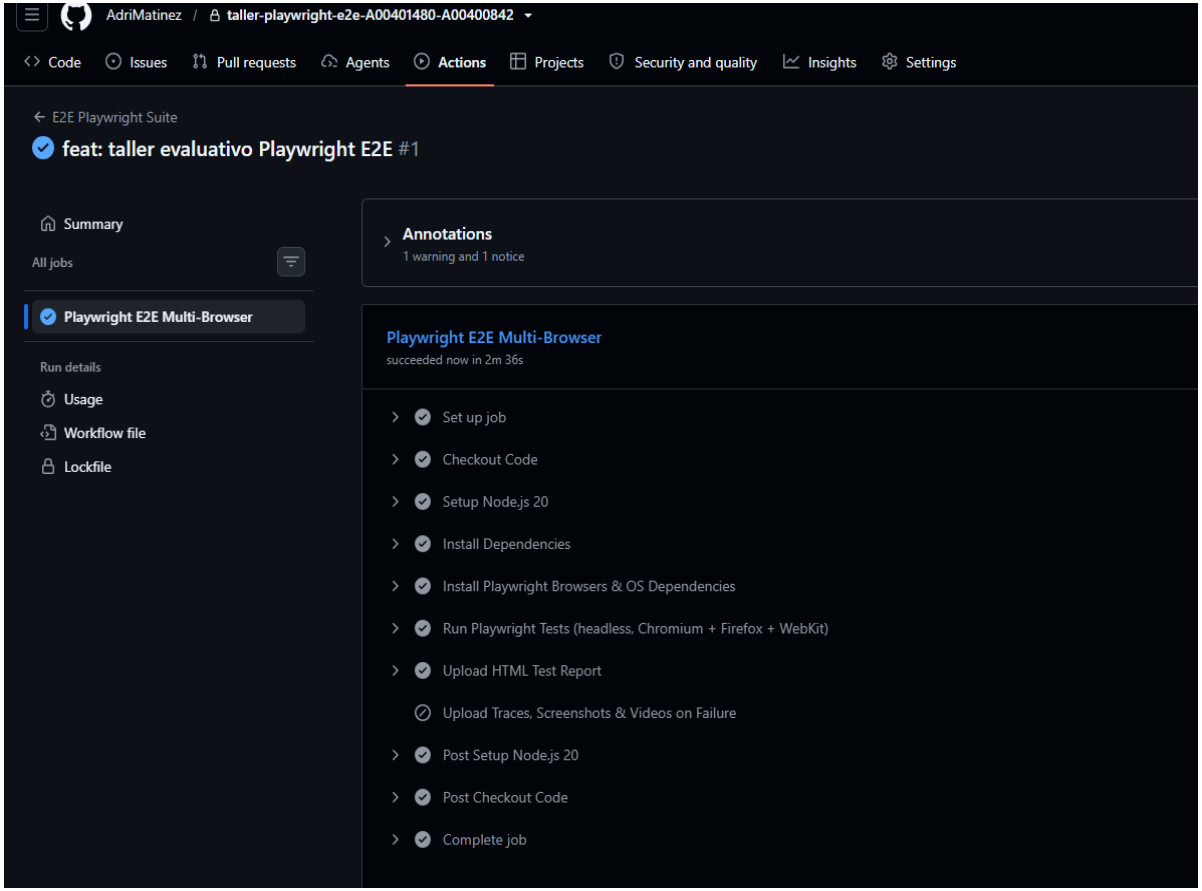


Figura 9. *Ejecución exitosa del workflow E2E Playwright Suite en GitHub Actions: job Playwright E2E Multi-Browser completado en 2 min 36 s.*

Punto 3 – Cuestionario de análisis conceptual y bitácora IAG

1. Localizadores semánticos frente a selectores frágiles: ¿Por qué se desaconseja XPath o CSS acoplados al DOM en favor de métodos ARIA (getByRole)? ¿Qué ventajas ofrecen ante refactorizaciones y en términos de accesibilidad WCAG?

Preferimos utilizar localizadores como getByRole() y getByLabel() porque nos permiten encontrar los elementos por su función o por el texto que los identifica, en lugar de depender de la estructura del HTML o de sus clases CSS. Por ejemplo, si cambiamos la distribución de un formulario o modificamos sus estilos, un XPath o un selector CSS puede dejar de funcionar, aunque la funcionalidad siga siendo la misma. Con los localizadores semánticos reducimos ese problema y hacemos que las pruebas sean más fáciles de mantener.

Además, estos localizadores nos ayudan a identificar posibles problemas de accesibilidad, porque los botones y campos deben tener nombres y roles que puedan reconocer las tecnologías de asistencia. Esto se relaciona con criterios WCAG como el 1.3.1, el 3.3.2 y el 4.1.2. Sin embargo, sabemos que utilizar estos métodos no garantiza por sí solo que la página cumpla con todos los requisitos de accesibilidad, por lo que también sería necesario realizar otras comprobaciones.

2. Aislamiento de contextos (BrowserContext): Compare la apertura de contextos en una sola instancia de Chromium frente a levantar procesos independientes (Selenium) en velocidad, memoria y aislamiento.

Con Playwright podemos crear varios BrowserContext dentro de una misma instancia de Chromium. Cada contexto mantiene su propio estado del navegador, incluyendo cookies y almacenamiento local, lo que permite ejecutar pruebas sin que compartan directamente la sesión. Esto suele reducir el tiempo y el consumo de recursos frente a crear procesos de navegador independientes para cada prueba.

En nuestro proyecto, este aislamiento es importante porque tenemos información de sesión y del carrito almacenada en el navegador. Así evitamos que una prueba herede datos de otra. Sin embargo, el aislamiento no es completo, ya que los contextos pueden compartir el mismo proceso del navegador y los datos del servidor siguen siendo compartidos. Por eso también debemos controlar los datos que creamos durante las pruebas para evitar conflictos.

3. Network Mocking frente a pruebas E2E puras: ¿Cuándo conviene interceptar llamadas con page.route() y cuándo es indispensable probar contra microservicios y bases de datos reales? ¿Qué riesgos acarrea abusar de los mocks?

Utilizamos page.route() cuando necesitamos simular situaciones específicas, como errores del servidor, retrasos en las respuestas o fallos de servicios externos. Esto nos permite comprobar cómo reacciona la interfaz sin tener que provocar esos problemas en el backend real.

Por otro lado, necesitamos realizar pruebas con los servicios reales para verificar que la aplicación funciona correctamente de principio a fin, incluyendo la comunicación con el backend, la autenticación, las validaciones y el procesamiento de los datos. Por eso, lo ideal es mantener los flujos principales conectados

a los servicios reales y utilizar mocks para probar situaciones difíciles de reproducir, como un error 500 o una respuesta lenta.

El principal riesgo de utilizar demasiados mocks es que las pruebas pueden pasar aunque exista un error en el sistema real, porque estamos comprobando una respuesta simulada. Para evitarlo, debemos combinar las pruebas simuladas con pruebas de integración y recorridos completos que utilicen los servicios reales.

4. Pruebas intermitentes (*Flaky Tests*): Explique cómo el *auto-waiting* de Playwright elimina las pausas fijas (sleep). ¿La política de *retries* en CI es una solución definitiva o paliativa?

El *auto-waiting* de Playwright nos ayuda a evitar pausas fijas como sleep o waitForTimeout, porque espera a que se cumplan las condiciones necesarias antes de realizar una acción. Por ejemplo, comprueba que un elemento esté visible, habilitado y listo para recibir una interacción. También podemos utilizar aserciones como toBeVisible() y toHaveText(), que vuelven a comprobar la condición hasta que se cumple o se alcanza el tiempo límite.

Consideramos que los *retries* en CI son una medida paliativa y no una solución definitiva. Aunque permiten que una prueba vuelva a ejecutarse cuando falla, no corrigen la causa del problema. Estos fallos pueden aparecer por problemas de sincronización, datos compartidos o servicios inestables. Por eso debemos revisar las pruebas que necesitan varios intentos, identificar la causa y corregirla. Los reintentos nos sirven para detectar y analizar estos casos, pero no deben utilizarse para ocultar errores.

5. Bitácora y uso ético de IAG (Nivel 3): ¿Qué herramientas de IA utilizamos para estructurar clases POM o *fixtures*? ¿Qué alucinaciones o errores detectamos y cómo los corregimos técnicamente?

Utilizamos la inteligencia artificial generativa como apoyo para comprender conceptos, organizar las pruebas y resolver dudas durante el desarrollo del taller. Nos puede servir para revisar la estructura del Page Object Model (POM), entender el uso de los *fixtures* y proponer alternativas para los localizadores y las aserciones.

Sin embargo, entendemos que las respuestas de la IA no siempre son correctas ni necesariamente se ajustan a nuestro proyecto. Por eso debemos revisar las sugerencias con la documentación de Playwright, comprobar que los métodos y localizadores coincidan con el código real y ejecutar las pruebas para verificar los resultados.

Para llevar una bitácora más completa, registramos las preguntas que hicimos, las sugerencias recibidas y las comprobaciones que realizamos. De esta manera, no solo mostramos que utilizamos una herramienta de IA, sino también cómo evaluamos sus respuestas y tomamos nuestras propias decisiones técnicas. Consideramos importante reconocer su uso y asegurarnos de comprender el código que entregamos, en lugar de aceptar las soluciones automáticamente.

Bitácora de interacción y validación de IAG

Pregunta o prompt que utilizamos	**Posible error o limitación de la respuesta de la IA**	**¿Cómo lo comprobamos técnicamente?**
¿Cómo podemos organizar las pruebas E2E utilizando el patrón Page Object Model en Playwright con TypeScript?	La IA propuso una estructura de carpetas o métodos que no coincidía con la organización de nuestro proyecto.	Revisamos los archivos existentes y comprobamos que las clases POM separen las interacciones con la interfaz de las pruebas y sus aserciones.
¿Cómo debemos localizar los campos de usuario, contraseña y el botón de inicio de sesión utilizando getByRole() y getByLabel()?	Sugirió nombres accesibles que no coincidían con los elementos reales de la página o utilizar un rol incorrecto.	Contrastamos los localizadores con el HTML y ejecutamos las pruebas para comprobar que encuentran los elementos correctos.
¿Cómo podemos comprobar que una orden se creó correctamente y que aparece el identificador generado por el servidor?	La IA asumió que el identificador es fijo o que la respuesta del servidor tuvo una estructura diferente a la real.	Revisamos la respuesta de la API y verificamos que la prueba compruebe el identificador generado, sin depender de un valor inventado o estático.
¿Cómo podemos utilizar page.route() para simular un error HTTP 500 y comprobar que la interfaz muestra un mensaje de error?	Sugirió una ruta que no coincidía con la petición real o una aserción que no compruebe el mensaje mostrado al usuario.	Verificamos la URL y el método de la petición, ejecutamos la prueba y comprobamos que la interfaz responda como esperamos.
¿Cómo podemos evitar waitForTimeout() y comprobar que el indicador de carga desaparece después de una respuesta lenta?	Propuso una espera fija que hizo la prueba más lenta sin garantizar que la carga haya terminado.	Utilizamos aserciones que esperan una condición concreta, como toBeVisible() y toBeHidden(), y revisamos los resultados de la prueba.

Análisis de la bitácora: Con estas comprobaciones entendemos que utilizar IA no significa aceptar todo el código que propone. La usamos como apoyo, pero la decisión final depende de que la solución corresponda con nuestro proyecto y funcione al ejecutar las pruebas. También reconocemos que un resultado exitoso no demuestra por sí solo que el código sea correcto en todos los escenarios, por lo que debemos revisar los casos cubiertos y las limitaciones de cada prueba.